SymPy Bug Report

1 Bug 10: Principal Cube-Root Inequality Includes Negative Reals

1.1 Minimal Reproducer

```
from sympy import S, N, symbols
from sympy.solvers.inequalities import solve_univariate_inequality

x = symbols("x")
sol = solve_univariate_inequality(x**(S(1)/3) <= 1, x, relational=False)

print("solution:", sol)
print("contains_minus_one:", sol.contains(-1))
print("lhs_at_minus_one:", N((-1)**(S(1)/3), 30))
```

1.2 Observed Behavior

```
solution: Interval(-oo, 1)
contains_minus_one: True
lhs_at_minus_one: 0.5 + 0.866025403784438646763723170753*I
```

SymPy reports that the solution set is $(-\infty, 1]$. In particular, the reported set contains $x = -1$, even though SymPy's principal value of $(-1)^{1/3}$ is non-real. Such a point cannot satisfy a real-valued inequality of the form $x^{1/3} \leq 1$.

1.3 Expected Behavior

The mathematically correct solution over the real line is

$$[0, 1].$$

More generally, for the parametrized family checked in the artifact,

$$\text{solve\_univariate\_inequality}\left(x^{1/3} \leq m,\ x,\ \texttt{relational=False}\right) = [0, m^3]$$

for the positive integer values $m = 1, \ldots, 6$.

1.4 Mathematical Explanation

SymPy's expression $x^{1/3}$, represented as `x**(S(1)/3)`, uses the principal branch of the power function. For a negative real input $x = -a$ with $a > 0$,

$$(-a)^{1/3} = \exp\left(\frac{\log(a) + i\pi}{3}\right) = a^{1/3}\left(\frac{1}{2} + \frac{\sqrt{3}}{2}i\right),$$

which is not real. Therefore the real-valued domain of the left-hand side is not all of $\mathbb{R}$; for this inequality it is $x \geq 0$.

On that domain, the principal cube root agrees with the ordinary nonnegative real cube root and is monotone increasing. Thus

$$x^{1/3} \leq 1 \quad \Longleftrightarrow \quad 0 \leq x \leq 1.$$

The returned interval $(-\infty, 1]$ is not an equivalent representation or a harmless branch convention: it contains points at which the left-hand side is complex, so the ordered real comparison is not satisfied.

1.5 Source-Level Diagnosis

The diagnosis identifies the primary source-level issue in `sympy/calculus/util.py`, in the function `continuous_domain`. The public call enters `solve_univariate_inequality` in `sympy/solvers/inequalities.py`. For the representative input, the solver forms $e = x^{1/3} - 1$, obtains the equality root $x = 1$, and asks `continuous_domain` for the real continuity domain of that expression. The interval-building code in `solve_univariate_inequality` then tests sample points between critical points and accepts whole intervals whose samples satisfy the inequality.

The problematic rule reported in `continuous_domain` treats rational powers with odd denominator as real-valued over all real bases:

```
if atom.exp.is_rational and den.is_odd:
    pass
```

That rule matches `real_root`-style semantics, but it is too broad for principal `Pow` semantics. For $x^{1/3} - 1$, it causes `continuous_domain` to return all real numbers rather than restricting the expression to $x \geq 0$. With the domain incorrectly set to $\mathbb{R}$, the inequality solver samples the interval $(-\infty, 1)$; the selected sample point satisfies the inequality, so the solver appends the entire interval $(-\infty, 1]$. No later filtering removes the negative inputs where the principal fractional power is complex.

The suggested fix direction in the diagnosis is to make `continuous_domain` distinguish principal `Pow` from real-root semantics. For rational non-integer powers of a real expression, a plausible fix is to require the base to be nonnegative unless SymPy can prove the principal value remains real on a larger set. The inequality solver should also intersect interval results with the real-valued domain of the relational expression.

1.6 Additional Failing Instances

The independent verification checks the representative case together with the positive integer parameters $m = 1, \ldots, 6$. In each case, SymPy returns an interval of the form $(-\infty, m^3]$, which includes negative real inputs such as -1. The expected behavior is uniformly $[0, m^3]$, because the principal power $x^{1/3}$ is real-valued only for $x \geq 0$, and on that domain $x^{1/3} \leq m$ is equivalent to $x \leq m^3$ for these positive m.

Input / Expression	SymPy behavior	Expected behavior
`solve_univariate_inequality`$(x^{1/3} \le 1)$	`Interval(-oo, 1)`	`Interval(0, 1)`
`solve_univariate_inequality`$(x^{1/3} \le 2)$	`Interval(-oo, 8)`	`Interval(0, 8)`
`solve_univariate_inequality`$(x^{1/3} \le 3)$	`Interval(-oo, 27)`	`Interval(0, 27)`
`solve_univariate_inequality`$(x^{1/3} \le 4)$	`Interval(-oo, 64)`	`Interval(0, 64)`
`solve_univariate_inequality`$(x^{1/3} \le 5)$	`Interval(-oo, 125)`	`Interval(0, 125)`
`solve_univariate_inequality`$(x^{1/3} \le 6)$	`Interval(-oo, 216)`	`Interval(0, 216)`

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest public material identified was SymPy documentation distinguishing principal roots from `real_root`, together with broader public awareness that principal-root domain handling can be subtle. The available evidence did not identify a clear public duplicate for this exact `solve_univariate_inequality` input or for the exact wrong interval `Interval(-oo, 1)`. This case is therefore individually actionable as a concrete regression test and issue, even though the general semantic pitfall is known.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Interval, S, symbols
from sympy.solvers.inequalities import solve_univariate_inequality

@pytest.mark.parametrize("m", [1, 2, 3, 4, 5, 6])
def test_fractional_power_inequality_real_domain(m):
    x = symbols("x")
    assert solve_univariate_inequality(x**(S(1)/3) <= m, x, relational=False) == Interval
    (0, m**3)
```